An exponential-hierarchy oracle separates worst-case derandomization from strong uniform hardness

Shuichi Hirahara, Zhenjian Lu, Hanlin Ren

doi:10.4230/LIPIcs.CCC.2023.6; CC BY 4.0

Editorial note. Complete official author proof transcribed from pages22–26. Original slice lengths, machine enumeration convention, probability thresholds, heavy-query claims, bounds and final oracle-computation algorithm retained. No mathematical repair or extra assumption is supplied.

Allowed standard prerequisites (editorial).

- BPP containment in the second level of the polynomial hierarchy. Original author Theorem23 [Sipser1983] retained; its approximate probability implementation is used as the stated standard prerequisite

Definitions and notations (original Sections 1.1, 2.1). For a complexity class $\mathfrak{C}$, a complexity-theoretic statement is $\mathfrak{C}$-relativizing if it is true relative to every oracle $\mathcal{O} \in \mathfrak{C}$.

Definition 20. A language L is in EXPH if there is a constant k, an exponential bound $e(n) = 2^{\mathrm{poly}(n)}$, and a $\mathrm{poly}(e(n))$-time algorithm M such that, for every input $x \in \{0,1\}^*$,

$$x \in L \iff \exists y_1 \forall y_2 \exists y_3 \forall y_4 \dots \exists y_k,\ M(x, y_1, \dots, y_k) = 1,$$

where $y_i \in \{0,1\}^{e(n)}$ for each $i \in [k]$. Equivalently, a language is in EXPH if it can be decided in exponential $(2^{\mathrm{poly}(n)})$ time with access to a PH oracle.

Pseudodeterministic algorithms. A pseudodeterministic algorithm is a probabilistic algorithm that returns a fixed output with high probability on any given input. We will also consider pseudodeterministic oracle (or advice-taking) algorithms. Such an algorithm will maintain its pseudodeterministic behavior (i.e., outputting a fixed answer with high probability for any input) when a correct oracle (or advice) is given.

Dense property (original Section 1.3.2). A property $Q \subseteq \{0,1\}^*$ is dense if for every input length $n \in \mathbb{N}$, we have that $|Q \cap \{0,1\}^n| \geq 2^n / \mathrm{poly}(n)$.

Theorem 23 [68]. For every oracle $\mathcal{O}$, $\mathsf{BPP}^{\mathcal{O}} \subseteq (\Sigma_2^p)^{\mathcal{O}}$.

Theorem 10. There is an oracle $\mathcal{O} \in \mathsf{EXPH}$ such that

$$\mathsf{RP}^{\mathcal{O}} \not\subseteq \text{i.o.}\,\mathsf{DTIME}^{\mathcal{O}}[2^n] \qquad \text{and} \qquad \mathsf{UP}^{\mathcal{O}} \not\subseteq \mathsf{BPTIME}^{\mathcal{O}}[2^n].$$

Proof of Theorem 10. Our oracle world will consist of two oracles P and Q (it is easy to combine them into a single oracle). The oracle P creates a hard problem in RP against deterministic algorithms, and the oracle Q creates a hard problem in UP against probabilistic algorithms. In particular, every input to the oracle P will be of the form (x, r) where $|r| = 100|x|$, and let

$$L_P := \{x : \exists r \in \{0,1\}^{100|x|}, P(x, r) = 1\}.$$

We will guarantee that for every x, $\Pr_r[P(x, r) = 1]$ is either 0 or greater than $1/2$, thus $L_P \in \mathsf{RP}$. We also define

$$L_Q := \{1^n : \exists r \in \{0,1\}^{100n}, Q(r) = 1\}.$$

We will guarantee that for every input length $100n$, $|Q \cap \{0,1\}^{100n}| \leq 1$, thus $L_Q \in \mathsf{UP}$. Let $M_1, M_2, \ldots$ be an enumeration of Turing machines running in deterministic 2^n time, and $N_1, N_2, \ldots$ be an enumeration of probabilistic Turing machines running in 2^n time (that does not necessarily satisfy the BPP promise).[1] We want every M_i to fail to solve L_P on all but finitely many input lengths, and every N_i to fail to solve L_Q on infinitely many input lengths. In particular, let n_1 be a large enough constant, and $n_i = 2^{500n_{i-1}}$ for each $i \geq 2$. Assuming that every machine appears in the sequence $\{N_i\}$ infinitely often, we want that for each $i \in \mathbb{N}$, N_i fails to solve L_Q on input 1^{n_i}. We also need the following terminologies. The n-th slice of P is the set of inputs of the form (x, r) where $|x| = n$ and $|r| = 100n$. The ℓ-th slice of Q is simply the set of inputs with length ℓ. During our construction, there will be some entries $P(x, r)$ and $Q(r)$ that are fixed, meaning that their values will never change in the subsequent construction. There will also be some inputs x that are frozen, meaning that for every $r \in \{0,1\}^{100|x|}$, the values of $P(x, r)$ are fixed and will never change in the subsequent construction. If an entry $P(x, r)$ or $Q(r)$ is not fixed and, in the case of $P(x, r)$, x is not frozen, then we say this entry is free. Our construction proceeds in stages. During stage n, we will fix the entire n-th slices of P and Q; we will possibly also fix or freeze some other entries. It will be guaranteed that before the n-th stage:

1. the number of frozen strings beyond the n-th slice (including the n-th slice) is at most $2^{n/2}$;
2. the number of fixed strings (except the frozen ones) beyond the n-th slice (including the n-th slice) is at most 2^{5n}; and
3. each fixed or frozen entry beyond the n-th slice (including the n-th slice) is set to be 0.

Diagonalization against M_i. Let $x_1, x_2, \ldots, x_n$ be n inputs of length n that are not frozen. Since there are at most $2^{n/2}$ frozen inputs of length n, it is always possible to select n unfrozen inputs. For each $i \in [n]$, we simulate the machine

[1] In this paper, whenever we enumerate a list of Turing machines, we assume every machine occurs in the list infinitely many times.

M_i on input x_i. Whenever M_i asks a query $P(x, r)$ or $Q(r)$, if this query is not free, then we return the corresponding value fixed before; if it is free, then we return 0 as the answer and also fix this query to be 0. Then M_i will output an answer $\mathsf{ans}_i \in \{0, 1\}$. Note that there are at most $2^{5n} + n \cdot 2^n \ll 2^{100n}/3$ entries in P that are fixed, thus the vast majority of the entries $P(x_i, r)$ are free. We fix all these entries $P(x_i, r)$ to be $(1 - \mathsf{ans}_i)$. It is easy to see that M_i fails to compute L_P on input x_i.

Diagonalization against N_i. If $n = 10n_i$ for some $i \in \mathbb{N}$, then we also need to fix the $100n_i$-th slice of Q so that N_i fails to compute L_Q on input 1^{n_i}. (In stage n, this happens after the above diagonalization against M_i.) We simulate N_i on input 1^{n_i}. Note that N_i is a probabilistic machine running in time 2^{n_i}, thus it has $B := 2^{2^{n_i}}$ computational branches, where each branch has $1/B$ probability mass. On each branch, whenever N_i asks a query $P(x, r)$ or $Q(r)$, if this query is not free, then we return the corresponding value fixed before; if it is free, then we return 0 as the answer to the query. Note that we cannot fix the query as there are potentially as many as B queries to fix. Instead, after simulating all the computational branches, we pick out the "heavy" queries that occur in a lot of branches and fix them:

- A string x is heavy if at least $\frac{1}{10^4|x|^3}$ fraction of branches queried some free entry of the form $P(x, r)$.
- An entry $Q(r)$ is heavy if at least $\frac{1}{10^4}$ fraction of branches queried $Q(r)$.

We freeze every heavy x by setting every unfixed entry $P(x, r)$ to be 0. We also fix every heavy $Q(r)$ to be 0. Note that each computational branch of N_i only makes at most 2^{n_i} queries. Therefore, for each input length m where $n \le m \le 2^{n_i}$, the number of heavy strings $x \in \{0, 1\}^m$ is at most $10^4 2^{n_i} m^3$. The number of entries $Q(r)$ fixed before this stage is at most 2^{5n}, and the number of heavy entries $Q(r)$ is at most $10^4 2^{n_i}$, therefore the total number of fixed entries in Q after this stage is at most $2^{6n} \ll 2^{100n_i}$. After simulating every computational branch of N_i, let p be the probability that N_i outputs 1 on input 1^n. If $p < 0.5$, then we pick a free $r \in \{0, 1\}^{100n_i}$ and set $Q(r) = 1$; since the number of fixed entries $Q(r)$ is $\ll 2^{100n_i}$, such r always exists. If $p \ge 0.5$, then we do nothing. Now we have that N_i solves L_Q on input 1^{n_i} with probability less than $2/3$ (actually, at most $1/2$). Is this always the case in the future? Consider how the future changes to the oracles might affect the behavior of N_i. Let m be an input length where $n < m \le 2^n$. During stage m (which has not happened yet), we will pick m inputs $x_1, x_2, \ldots, x_m \in \{0, 1\}^m$, then some entries of the form $P(x_i, r)$ will be set to be 1. Also note that the probability mass of computational branches that each x_i affects is at most $\frac{1}{10^4 m^3}$. Therefore, the total probability mass of computational branches affected is at most

$$\sum_{m \in \mathbb{N}} \frac{m}{10^4 m^3} < \frac{1}{10^3}.$$

Note that we also set at most one entry $Q(r)$ to be 1, where $|r| = 100n_i$. (The next time we diagonalize against N_{i+1} happens in stage $n_{i+1} \gg 2^n$.) Since this entry $Q(r)$ is not heavy, the probability mass of computational branches that it affects is at most $\frac{1}{10^4}$. It follows that only an $\frac{11}{10^4}$ fraction of computational

branches of N_i will be affected in the future. That is, let p' be the probability that N_i outputs 1 on input 1^{n_i} at the end of our construction (i.e., after stage $\gg 2^n$), then $|p' - p| < \frac{11}{10^4}$. Therefore, it cannot be the case that

$$\Pr[N_i(1^{n_i}) = L_Q(1^{n_i})] \geq 2/3.$$

Clean-up. At the end of stage n, we fix every unfixed input $P(x, r)$ in the n-th slice to be 0. If $n = 10n_i$ then we also fix every unfixed input $Q(r)$ in the $10n$-th slice to be 0. The number of frozen strings is at most $10^4 2^{n/10} n^3 \leq 2^{n/2}$ (note that at most one N_i contributes to the frozen strings since the input lengths $\{n_i\}$ are very far apart). The number of fixed entries above the $(n+1)$-st slice is at most

$$2^{5n} + n \cdot 2^n + \sum_{m=n}^{2^{n/10}} 10^4 2^{n/10} m^3 < 2^{5(n+1)}.$$

The complexity of $\mathcal{O}$. We show that the above oracles P and Q are computable in EXPH. It suffices to present a deterministic algorithm with access to a Σ_3^p oracle that on input 1^n, outputs the truth tables of the n-th slices of P and Q in $2^{O(n)}$ time. We simulate each stage $n' \leq n$. Before stage n', we maintain the set of fixed entries $P(x, r), Q(r)$, as well as the set of frozen strings x. There are at most $2^{O(n')}$ many such strings. In what follows, we use $\langle P, Q\rangle$ to denote (the length-$2^{O(n')}$ encoding of) the list of fixed entries and frozen strings. It is easy to diagonalize against each M_i: we simply choose the inputs $x_1, x_2, \ldots, x_{n'}$, simulate each M_i on input x_i, and fix the oracles accordingly. This takes $2^{O(n')}$ time. To simulate N_i, we need to define the following language capturing the weight of queries. Consider simulating the probabilistic machine N_i on the current oracle $\langle P, Q\rangle$. Whenever N_i makes a query, if this query is not free, then we return the answer of the query; otherwise we return 0. The following two claims are immediate corollaries of Theorem 23:

Claim 42. There is a language $\mathsf{Heavy\text{-}String}(\langle P, Q\rangle, N_i, x)$ computable in Σ_2^p such that:

- if at least $\frac{1.1}{10^4|x|^3}$ fraction of computational branches of N_i queried some free entry of the form $P(x, -)$, then $\mathsf{Heavy\text{-}String}(\langle P, Q\rangle, N_i, x) = 1$;
- if at most $\frac{0.9}{10^4|x|^3}$ fraction of computational branches of N_i queried some free entry of the form $P(x, -)$, then $\mathsf{Heavy\text{-}String}(\langle P, Q\rangle, N_i, x) = 0$.

Claim 43. There is a language $\mathsf{Heavy\text{-}Entry}(\langle P, Q\rangle, N_i, r)$ computable in Σ_2^p such that:

- if at least $\frac{1.1}{10^4}$ fraction of branches of N_i queried $Q(r)$, then $\mathsf{Heavy\text{-}Entry}(\langle P, Q\rangle, N_i, r) = 1$;
- if at most $\frac{0.9}{10^4}$ fraction of branches of N_i queried $Q(r)$, then $\mathsf{Heavy\text{-}Entry}(\langle P, Q\rangle, N_i, r) = 0$.

(Note that both $\mathsf{Heavy\text{-}String}$ and $\mathsf{Heavy\text{-}Entry}$ accepts inputs of length $2^{O(n')}$.)
With the aid of these two languages, it is possible to simulate stage n' now. We

say a string x is heavy if $\mathsf{Heavy\text{-}String}(\langle P, Q\rangle, N_i, x) = 1$, and an entry $Q(r)$ is heavy if $\mathsf{Heavy\text{-}Entry}(\langle P, Q\rangle, N_i, r) = 1$. Since there are at most $2^{O(n')}$ heavy strings, we can find all of them in $2^{O(n')}$ time with access to an $\mathsf{NP}^{\mathsf{Heavy\text{-}String}}$ oracle. Similarly, since there are $2^{O(n')}$ heavy entries, we can find all of them in $2^{O(n')}$ time with access to an $\mathsf{NP}^{\mathsf{Heavy\text{-}Entry}}$ oracle. We freeze these heavy strings and fix these heavy entries accordingly. Finally, we estimate the probability that N_i outputs 1 (given current versions of P and Q) in Σ_2^p, within additive error 0.1. If this estimation is less than $1/2$, then we pick a free $r \in \{0,1\}^{10n'}$ and set $Q(r) = 1$; otherwise we do nothing. At the end of stage n', we fix every unfixed entry in the n'-th slice of $P(x, r)$ to be 0. If $n = 10n_i$ then we also fix every unfixed input $Q(r)$ in the $10n$-th slice to be 0. This takes $2^{O(n')}$ time. It is easy to see that simulating the n stages takes $2^{O(n)}$ time with access to a Σ_3^p oracle. □

References

[31] Shafi Goldwasser, Russell Impagliazzo, Toniann Pitassi, and Rahul Santhanam. On the pseudo-deterministic query complexity of NP search problems. CCC2021, LIPIcs200,36:1–36:22. doi:10.4230/LIPIcs.CCC.2021.36.

[33] Shuichi Hirahara. Non-black-box worst-case to average-case reductions within NP. FOCS2018,247–258. doi:10.1109/FOCS.2018.00032.

[34] Shuichi Hirahara. Non-disjoint promise problems from meta-computational view of pseudorandom generator constructions. CCC2020,LIPIcs169,20:1–20:47. doi:10.4230/LIPIcs.CCC.2020.20.

[68] Michael Sipser. A complexity theoretic approach to randomness. STOC1983,330–335. doi:10.1145/800061.808762.